# System Design: Real-Life Usage Guide

## Table of Contents



---

## Load Balancing - Real World Usage

### When Do You Need Load Balancing?

🚨 **Signs You Need It:**

- Your server crashes during peak hours
- Users complain about slow response times
- CPU usage constantly above 80%
- Getting 500+ users at the same time

💡 **Real-Life Example:**

**E-commerce Website During Sale**

- Normal day: 100 users → 1 server works fine
- Sale day: 10,000 users → 1 server crashes
- **Solution**: Add 5 servers + load balancer

```
Before: [Users] → [Single Server] → [Database]
After:  [Users] → [Load Balancer] → [Server 1]
                                  → [Server 2]
                                  → [Server 3]
```

**How to Implement (Simple Steps):**

**Step 1: Choose Load Balancer Type**

- **Small website**: Use cloud load balancer (AWS ALB, Google Cloud LB)
- **Medium website**: Use Nginx or HAProxy
- **Large website**: Use multiple layers

**Step 2: Pick Algorithm**

- **Round Robin**: For similar servers
- **Least Connections**: For chat apps, video calls
- **IP Hash**: For shopping carts (same user → same server)

**Step 3: Set Up Health Checks**

```
Every 30 seconds, check:
- Is server responding?
- Response time under 2 seconds?
- Error rate under 5%?
```

**Real Examples:**

- **Netflix**: Uses load balancers to distribute video streaming
- **Instagram**: Balances photo uploads across servers
- **Uber**: Balances ride requests across different regions

---

## Caching - When and How to Use

**When to Use Caching?**

🚨 **Signs You Need It:**

- Database queries taking more than 1 second
- Same data requested multiple times
- Users see loading screens frequently
- High database CPU usage

💡 **Real-Life Scenarios:**

**1. News Website**

- **Problem**: Latest news fetched from database every time
- **Solution**: Cache news for 5 minutes
- **Result**: 90% faster loading

**2. Social Media Feed**

- **Problem**: User timeline takes 3 seconds to load
- **Solution**: Cache user's timeline for 10 minutes
- **Result**: Instant feed loading

**3. E-commerce Product Page**

- **Problem**: Product details queried every visit
- **Solution**: Cache product info for 1 hour
- **Result**: Fast product browsing

**How to Implement (Step by Step):**

**Step 1: Choose Cache Type**

```
Static Files (images, CSS): Use CDN
Database Results: Use Redis/Memcached
API Responses: Use application cache
User Sessions: Use memory cache
```

**Step 2: Set TTL (Time to Live)**

```
User Profile: 30 minutes (changes rarely)
Product Prices: 5 minutes (changes often)
News Articles: 15 minutes (moderately changes)
Stock Prices: 1 minute (changes frequently)
```

**Step 3: Cache Strategy**

```python
```

```
# Simple caching example
def get_user_profile(user_id):
    # Check cache first
    cached_profile = cache.get(f"user_{user_id}")
    if cached_profile:
        return cached_profile

    # If not in cache, get from database
    profile = database.get_user(user_id)

    # Store in cache for 30 minutes
    cache.set(f"user_{user_id}", profile, 1800)
    return profile
```

**Real Examples:**

- **Facebook**: Caches your timeline, friend lists
- **YouTube**: Caches popular videos closer to users
- **Amazon**: Caches product recommendations

---

## Database Design - Practical Decisions

### When to Use SQL vs NoSQL?

**Use SQL When:**

- **Banking App**: Need ACID transactions
- **Inventory System**: Complex relationships between products, orders, customers
- **Accounting Software**: Data must be consistent and accurate
- **CRM System**: Structured data with relationships

**Use NoSQL When:**

- **Social Media**: Flexible post formats (text, images, videos)
- **IoT Data**: Millions of sensor readings
- **Content Management**: Articles, blogs with varying structures
- **Real-time Analytics**: Fast reads/writes needed

**Real-Life Decision Making:**

**Scenario 1: Online Store**

Products → Categories → Orders → Customers
(All connected, need consistency)
Decision: Use SQL (PostgreSQL/MySQL)

**Scenario 2: Chat Application**

Messages: { user_id, message, timestamp, room_id }
(Simple structure, need speed)
Decision: Use NoSQL (MongoDB)

**Scenario 3: Multi-tenant SaaS**

Each customer has different data fields
Decision: Use NoSQL for flexibility

**Database Optimization in Practice:**

**When to Add Indexes:**

- **Sign**: Query taking >1 second
- **Action**: Add index on frequently searched columns
- **Example**: User login → index on email field

**When to Partition:**

- **Sign**: Database size >100GB
- **Action**: Split by date, user region, or category
- **Example**: Order history → partition by year

**When to Add Read Replicas:**

- **Sign**: Database CPU >80% due to reads
- **Action**: Send reads to replica, writes to master
- **Example**: Analytics queries → read replica

---

## Microservices - When to Break Apart

**When to Move from Monolith to Microservices?**

🚨 **Signs You Need Microservices:**

- Different teams stepping on each other's code
- Deployment takes hours and affects entire system
- One feature down = entire app down
- Different parts need different technologies
- Team size >8-10 people

💡 **Real-Life Breakdown Example:**

**E-commerce Monolith → Microservices**

```
Before: One Big App
- User management
- Product catalog
- Order processing
- Payment handling
- Notification system

After: Separate Services
- User Service (handles login, profiles)
- Product Service (manages catalog)
- Order Service (processes orders)
- Payment Service (handles payments)
- Notification Service (sends emails/SMS)
```

**How to Break Apart (Step by Step):**

**Step 1: Identify Boundaries**

```
Look for:
- Different data models
- Different business rules
- Different change frequencies
- Different team responsibilities
```

**Step 2: Start Small**

```
Don't break everything at once!
1. Start with least critical service
2. Test thoroughly
3. Move to next service
4. Repeat
```

**Step 3: Handle Communication**

```
Synchronous: User Service → Product Service (get product details)
Asynchronous: Order Service → Notification Service (send order confirmation)
```

**Real Examples:**

- **Netflix**: Started as monolith, now 500+ microservices
- **Uber**: Different services for riders, drivers, payments
- **Amazon**: Each team owns their service

---

## Message Queues - Real Examples

### When to Use Message Queues?

### 🚨 Signs You Need Them:

- Tasks taking too long and blocking users
- Services going down and losing data
- Need to process things in background
- Different services need to communicate

### 💡 Real-Life Use Cases:

**1. Email Sending**

```
Problem: User registration takes 5 seconds (sending welcome email)
Solution: Queue the email, return success immediately
Result: User sees instant success, email sent in background
```

**2. Image Processing**

```
Problem: Photo upload takes 30 seconds to resize
Solution: Queue image processing, show "processing..." to user
Result: User can continue using app while image processes
```

**3. Payment Processing**

Problem: Payment service down = entire checkout broken
Solution: Queue payment requests, process when service returns
Result: Orders never lost, payments processed when possible

**How to Implement:**

### Step 1: Choose Queue Type

Simple Tasks: Redis Queue
High Volume: Apache Kafka
Reliability: RabbitMQ
Cloud: AWS SQS, Google Pub/Sub

### Step 2: Set Up Producer

python

```python
# Example: Order processing
def create_order(order_data):
    # Save order to database
    order = database.save_order(order_data)

    # Queue background tasks
    queue.send('send_confirmation_email', order.id)
    queue.send('update_inventory', order.items)
    queue.send('notify_shipping', order.id)

    return order
```

### Step 3: Set Up Consumers

python

```python
# Email worker
def process_email(order_id):
    order = database.get_order(order_id)
    email_service.send_confirmation(order.user_email)

# Inventory worker
def update_inventory(items):
    for item in items:
        database.reduce_stock(item.id, item.quantity)
```

**Real Examples:**

- **Instagram**: Queues photo processing, feed updates
- **Slack**: Queues message delivery, notifications
- **Shopify**: Queues order processing, inventory updates

---

## CDN - When Your Site Gets Slow

### When to Use CDN?

### 🚨 Signs You Need CDN:

- Users far from your server complain of slow loading
- Images/videos take long to load
- Website slow in different countries
- High bandwidth costs

### 💡 Real-Life Scenarios:

**1. Global News Website**

```
Problem: Users in Asia wait 5 seconds for images from US server
Solution: CDN caches images in Asian data centers
Result: Images load in 0.5 seconds for Asian users
```

**2. Online Course Platform**

```
Problem: Video lectures buffer frequently
Solution: CDN caches videos worldwide
Result: Smooth video playback globally
```

### How to Implement:

**Step 1: Choose CDN Provider**

```
Budget-friendly: Cloudflare (free tier available)
Performance: AWS CloudFront
Global reach: Google Cloud CDN
```

**Step 2: Configure Caching Rules**

```
Images/Videos: Cache for 1 year
CSS/JavaScript: Cache for 1 month
HTML pages: Cache for 1 hour
API responses: Don't cache (or very short)
```

**Step 3: Update Your Code**

```html
<!-- Before -->
<img src="https://mysite.com/images/logo.png">

<!-- After -->
<img src="https://cdn.mysite.com/images/logo.png">
```

**Real Examples:**

- **Netflix**: CDN for video streaming worldwide
- **Spotify**: CDN for music streaming
- **GitHub**: CDN for code repositories

---

## Scaling - Step by Step Process

**The Real-World Scaling Journey:**

**Stage 1: Single Server (0-1,000 users)**

```
Setup: [Users] → [Single Server] → [Database on same server]
Cost: $50/month
When to scale: Server CPU >80%, response time >2 seconds
```

**Stage 2: Separate Database (1,000-10,000 users)**

```
Setup: [Users] → [App Server] → [Database Server]
Cost: $200/month
When to scale: Database becomes bottleneck
```

**Stage 3: Add Load Balancer (10,000-100,000 users)**

```
Setup: [Users] → [Load Balancer] → [Multiple App Servers] → [Database]
Cost: $500/month
When to scale: Database reads slow
```

**Stage 4: Add Cache & Read Replicas (100,000-1M users)**

```
Setup: [Users] → [Load Balancer] → [App Servers] → [Cache] → [Master DB]
                                                 → [Read Replicas]
Cost: $2,000/month
When to scale: Database writes slow
```

**Stage 5: Microservices & Sharding (1M+ users)**

```
Setup: Multiple services with their own databases
Cost: $10,000+/month
```

**Real Scaling Example - Instagram:**

```
2010: 1 server, 1 database
2011: Load balancer + multiple servers
2012: Redis cache + database replicas
2013: Separate photo service
2014: Microservices architecture
2015: Database sharding
```

---

## Security - Practical Implementation

**When to Implement Each Security Measure:**

**Day 1: Basic Security**

```
1. HTTPS everywhere
2. Strong password requirements
3. Basic input validation
4. SQL injection prevention
```

**Week 1: Authentication**

1. JWT tokens for API
2. Session management
3. Password hashing (bcrypt)
4. Login attempt limiting

### Month 1: Advanced Security

1. Rate limiting per user
2. API key management
3. Data encryption at rest
4. Security headers

### Real-Life Security Implementation:

### 1. User Registration System

python

```python
# Basic security example
def register_user(email, password):
    # Input validation
    if not is_valid_email(email):
        return "Invalid email"

    if len(password) < 8:
        return "Password too short"

    # Check for existing user
    if user_exists(email):
        return "User already exists"

    # Hash password
    hashed_password = bcrypt.hash(password)

    # Save to database
    user = create_user(email, hashed_password)
    return user
```

### 2. API Rate Limiting

python

```
# Prevent API abuse
def rate_limit_check(user_id):
    key = f"rate_limit:{user_id}"
    current_requests = cache.get(key) or 0

    if current_requests >= 100:  # 100 requests per hour
        return False

    cache.set(key, current_requests + 1, 3600)  # 1 hour
    return True
```

**Real Examples:**

- **Bank Apps**: Multi-factor authentication, encryption
- **Social Media**: Content filtering, privacy controls
- **E-commerce**: Fraud detection, secure payments

---

## Monitoring - What to Watch

### What to Monitor at Each Stage:

### Stage 1: Small App (Basic Monitoring)

```
- Server CPU, memory, disk usage
- Application response time
- Error rate
- Database connection count
```

### Stage 2: Growing App (Detailed Monitoring)

```
- User activity metrics
- API endpoint performance
- Database query performance
- Cache hit rates
```

### Stage 3: Large App (Advanced Monitoring)

```
- Business metrics (revenue, conversions)
- User experience metrics
- Security events
- Infrastructure costs
```

**Real-Life Monitoring Setup:**

### 1. Set Up Alerts

```
Critical (Wake up at night):
- Server down
- Error rate >5%
- Response time >5 seconds

Important (Check during work):
- CPU usage >80%
- Memory usage >85%
- Disk space >90%

Info (Weekly review):
- Slow queries
- Cache performance
- User activity trends
```

### 2. Create Dashboards

```
Executive Dashboard:
- Active users
- Revenue metrics
- System uptime

Technical Dashboard:
- Response times
- Error rates
- Infrastructure health

Business Dashboard:
- User engagement
- Feature usage
- Conversion rates
```

**Real Examples:**

- **Airbnb**: Monitors booking completion rates
- **Uber**: Monitors ride completion times
- **Netflix**: Monitors video streaming quality

---

## Real Project Examples

### Example 1: Building a Chat Application

**Requirements:**

- 10,000 users online simultaneously
- Messages delivered instantly
- Message history stored
- Group chats supported

**System Design Decisions:**

**1. Database Choice:**

```
Messages: NoSQL (MongoDB) - flexible schema
Users: SQL (PostgreSQL) - structured data
Reason: Messages have simple structure, users need relationships
```

**2. Real-time Communication:**

```
Technology: WebSockets
Why: Instant message delivery needed
Alternative: Server-sent events (one-way communication)
```

**3. Scaling Strategy:**

```
Stage 1: Single server with WebSocket connections
Stage 2: Add Redis for storing active connections
Stage 3: Multiple servers with Redis pub/sub
Stage 4: Message queue for reliable delivery
```

**4. Caching Strategy:**

```
Recent messages: Cache in Redis for 1 hour
User online status: Cache in memory
Chat rooms: Cache for 30 minutes
```

### Example 2: E-commerce Platform

**Requirements:**

- 100,000 products

- 50,000 daily orders
- Global customers
- 99.9% uptime

**System Design Decisions:**

**1. Database Design:**

```
Products: SQL database (complex relationships)
Orders: SQL database (ACID properties needed)
User sessions: NoSQL (fast reads/writes)
Analytics: NoSQL (large volume data)
```

**2. Caching Strategy:**

```
Product catalog: CDN + Redis (1 hour TTL)
User cart: Redis (24 hour TTL)
Search results: Redis (15 minutes TTL)
```

**3. Scaling Plan:**

```
Week 1: Single server + database
Month 1: Add load balancer + read replicas
Month 3: Add CDN for images
Month 6: Microservices (user, product, order services)
Year 1: Database sharding by region
```

**4. Security Implementation:**

```
Day 1: HTTPS, input validation
Week 1: User authentication (JWT)
Month 1: Payment security (PCI compliance)
Month 3: Rate limiting, DDoS protection
```

**Example 3: Social Media Platform**

**Requirements:**

- 1 million users
- Real-time feed updates
- Photo/video sharing
- Global reach

**System Design Decisions:**

**1. Database Strategy:**

```
User profiles: SQL (PostgreSQL)
Posts/comments: NoSQL (MongoDB)
Media files: Object storage (AWS S3)
Feed timeline: Redis cache
```

**2. Feed Generation:**

```
Strategy: Push model for active users, pull for inactive
Implementation: Message queue processes new posts
Cache: Pre-computed feeds in Redis
```

**3. Media Handling:**

```
Upload: Direct to S3 with signed URLs
Processing: Queue-based image/video processing
Delivery: CDN for global fast access
```

**4. Real-time Features:**

```
Notifications: WebSockets + message queues
Live updates: Server-sent events
Activity feeds: Redis pub/sub
```

---

## Quick Decision Framework

**When You're Building Something New:**

**Step 1: Estimate Scale**

```
<1,000 users: Single server
1,000-10,000 users: Add load balancer
10,000-100,000 users: Add caching
100,000+ users: Consider microservices
```

**Step 2: Choose Database**

Structured data + relationships: SQL
Flexible schema + high volume: NoSQL
Need both: Use both (polyglot persistence)

**Step 3: Plan for Growth**

Month 1: Build simple, working version
Month 3: Add monitoring and basic scaling
Month 6: Optimize based on real usage
Year 1: Major architecture changes if needed

**Step 4: Security Checklist**

✓ HTTPS everywhere
✓ Input validation
✓ Authentication system
✓ Rate limiting
✓ Data encryption
✓ Regular security audits

Remember: **Start simple, measure everything, scale based on real problems, not imaginary ones!**

The key is to implement these concepts when you actually need them, not before. Most startups fail because they over-engineer from day one instead of focusing on solving real user problems.